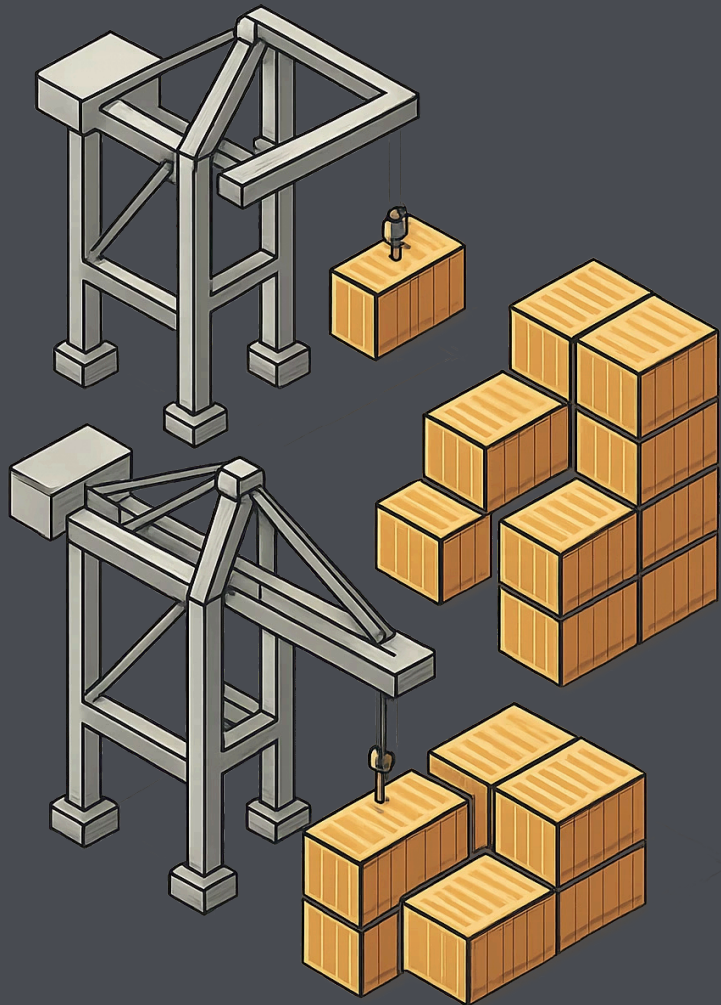




Day 3: Docker Deep Dive & Kubernetes Intro

Mastering Docker internals, multi-container orchestration with Docker Compose, and your first steps into Kubernetes — all in one session.

Sami Selim

Instructor: DevOps Engineering

DAY 3

INTERMEDIATE-ADVANCED

Today's Agenda

A full-stack tour from Docker fundamentals to cluster orchestration. Here's what we'll cover:

01

Docker Commands Review

The most common commands every engineer should know cold.

03

Docker Volumes

Persistent data strategies — managed volumes and bind mounts.

05

Docker Compose

Declare and orchestrate multi-container environments with a single YAML file.

02

Docker Networking

How containers find and talk to each other across isolated networks.

04

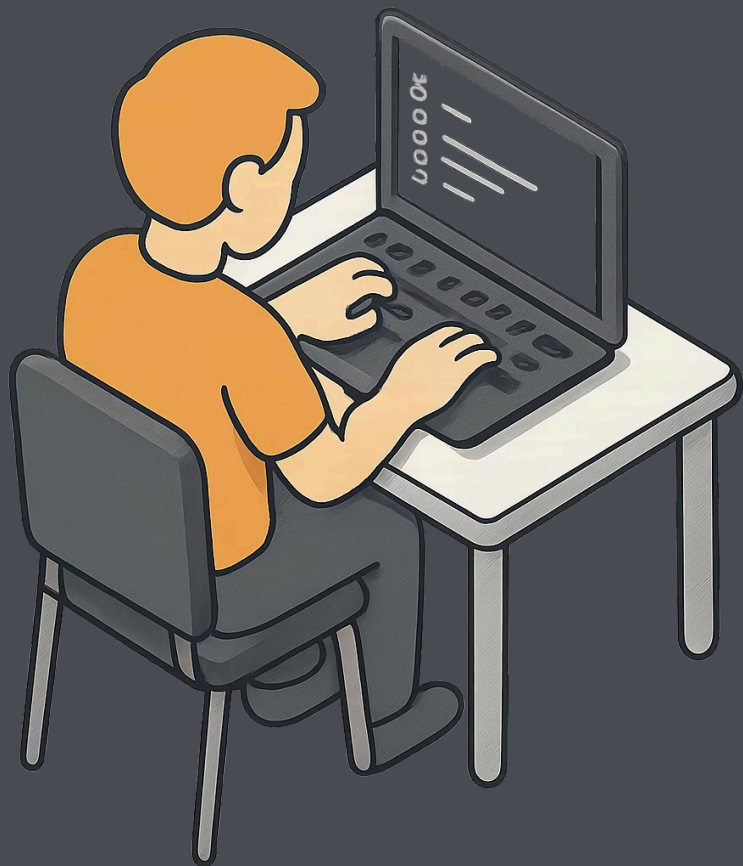
Dockerfile Deep Dive

Advanced instructions, CMD vs ENTRYPOINT, COPY vs ADD, and best practices.

06

Intro to Kubernetes (K8s)

Architecture, core objects, Pods, Deployments, and the kubectl CLI.



SECTION 1

Docker Commands Review

The Daily Toolkit

Container Lifecycle Commands

These are the bread-and-butter commands you'll reach for dozens of times a day. Master them and you'll move through any Docker environment with confidence.

```
docker run -d -p 80:80 nginx
```

Runs an Nginx container in detached (background) mode, mapping port 80 on your host to port 80 inside the container. The `-d` flag is essential for long-running services.

```
docker ps / docker ps -a
```

Lists all currently running containers. Add `-a` to include stopped containers — invaluable when debugging a container that exited unexpectedly.

```
docker stop / docker start
```

Gracefully stops a running container (sends SIGTERM), or restarts a stopped one. Always prefer `stop` over `kill` to allow proper shutdown hooks.

```
docker rm <container_id>
```

Removes a stopped container permanently. Append `-f` to force-remove a running container — use with care in production.

Image & Troubleshooting Commands

Beyond lifecycle management, these commands are your diagnostic toolkit — from inspecting images to live-streaming logs and jumping inside a running container.

`docker images`

Lists all images stored locally, including their tag, image ID, and size on disk.

`docker rmi <image_id>`

Deletes a local image. Useful for reclaiming disk space after testing multiple image versions.

`docker logs -f`

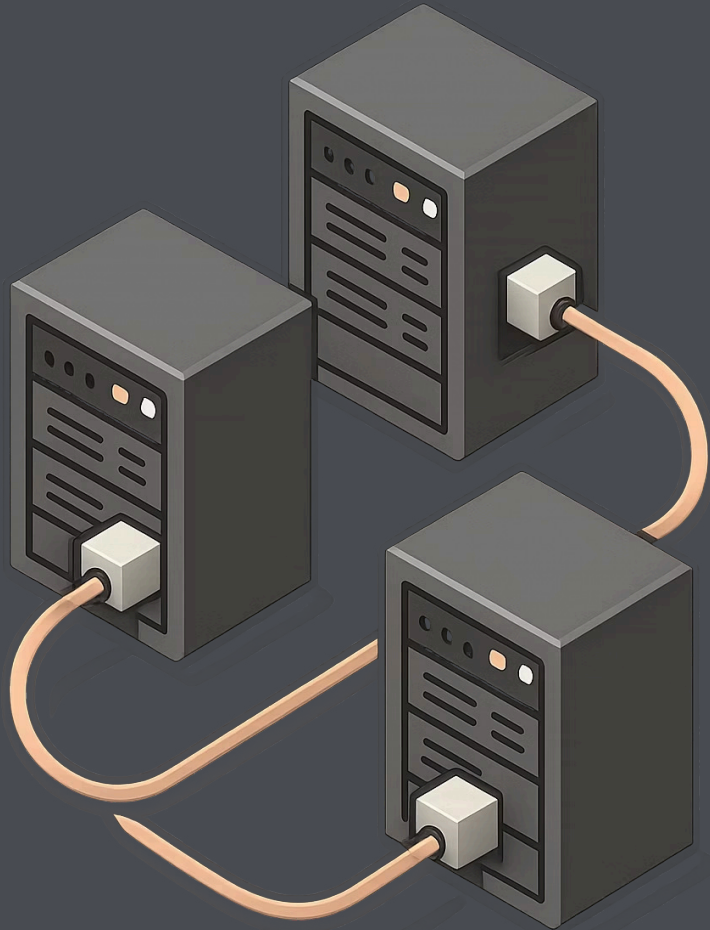
Streams real-time stdout/stderr output from a container. The `-f` flag tails the log continuously — perfect for live debugging.

`docker exec -it ... /bin/sh`

Opens an interactive shell inside a running container. The most powerful debugging tool in your arsenal — inspect the filesystem, run commands, and inspect processes live.

`docker system prune`

Cleans up all stopped containers, unused networks, and dangling images in one sweep. Run this regularly to keep your disk healthy.



SECTION 2

Docker Networking

How Do Containers Talk?

Container Communication & Network Modes

Docker's networking model is one of its most powerful — and most misunderstood — features. Understanding it unlocks seamless container-to-container communication.

Default Bridge Network

Every Docker installation ships with a built-in virtual bridge network named `bridge`. Containers on the default bridge can talk by IP only — no automatic DNS. Good for quick tests, but not for production workflows.

Custom Networks

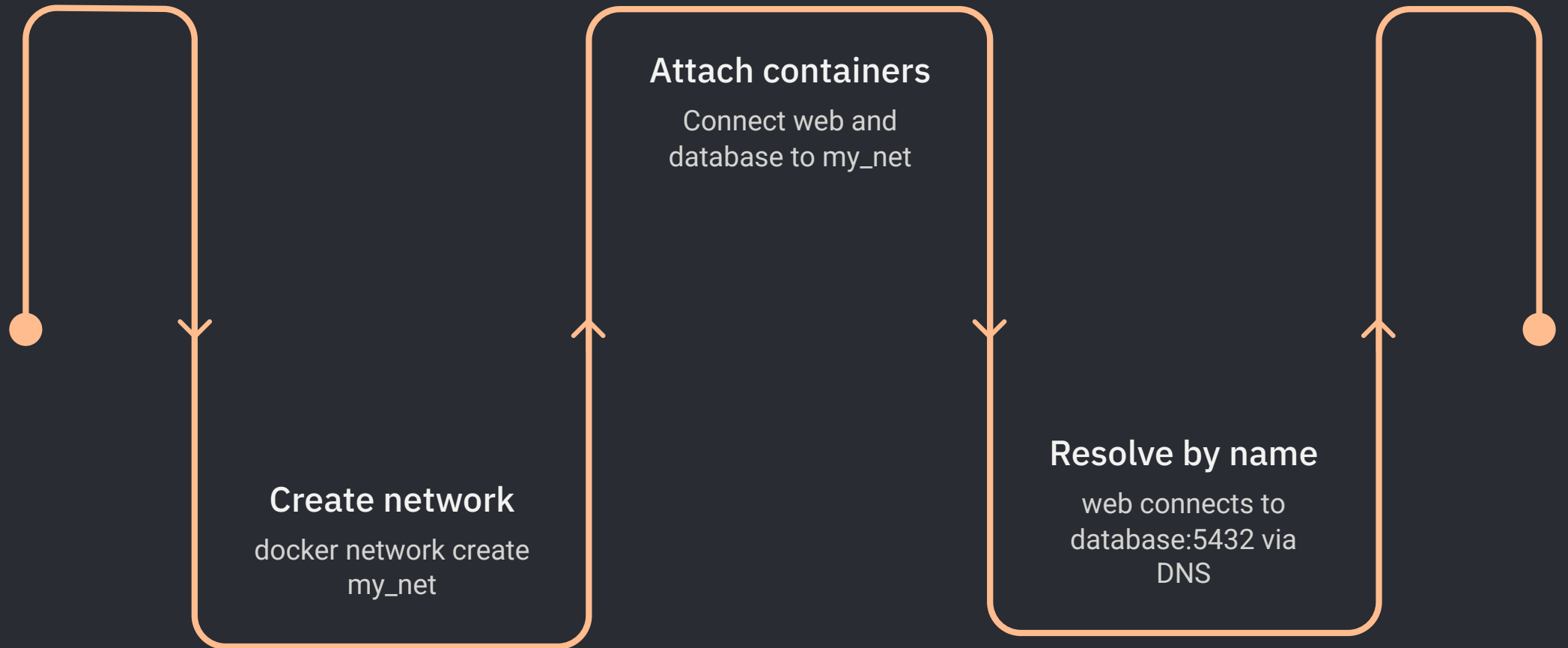
Create your own isolated network with `docker network create my_net`. Containers attached to the same custom network get automatic DNS resolution by container name.

Example: A `web` container can reach a `database` container simply by connecting to `http://database:5432` — no IP lookups needed.

Host Network

Maps the container directly to the host's network stack. Maximum performance, zero network isolation. Use only when you know exactly what you're doing.

Network DNS in Action



Custom networks are the recommended approach for any multi-container setup. DNS-based discovery eliminates hardcoded IPs, making your architecture far more portable and maintainable.

SECTION 3

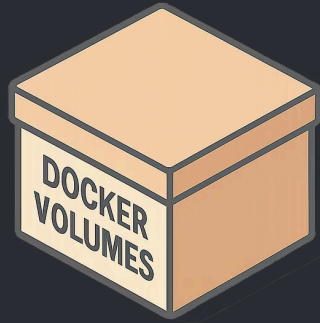
Docker Volumes & Persistent Data

The Ephemeral Problem



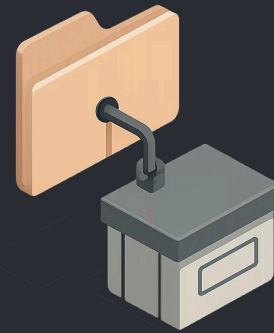
Why Containers Lose Data — and How to Fix It

By default, a container's filesystem is completely ephemeral. The moment you run `docker rm`, every file written inside that container is gone forever. For stateless apps that's fine — for databases, it's a disaster. Docker gives you two solutions.



Managed Volumes

Docker-managed storage, typically living at `/var/lib/docker/volumes/` on the host. Data persists even after the container is destroyed and removed. The recommended approach for databases like PostgreSQL and MySQL — Docker handles all the path management for you.



Bind Mounts

Maps a specific, absolute path on your host machine (e.g., `/Users/dev/app`) directly into the container filesystem. Changes on the host appear instantly inside the container — ideal for local development with live-reloading code. The tradeoff: it's tightly coupled to your host directory structure.

Volume Commands & Usage

Docker gives you a full set of CLI commands to create, inspect, and manage volumes independently of any container lifecycle.

1

```
docker volume create my_data
```

Explicitly creates a named volume. You can also let Docker create it automatically on first use.

2

```
docker volume ls
```

Lists all volumes currently managed by Docker on this host.

3

```
docker volume inspect my_data
```

Reveals the volume's physical location on the host filesystem, creation date, and driver details.

4

Mount in `docker run`

Volume: `-v my_data:/var/lib/postgresql/data`

Bind Mount: `-v /home/user/app:/app`

SECTION 4

Dockerfile Deep Dive Beyond the Basics



Core Dockerfile Instructions

A Dockerfile is the precise blueprint Docker uses to build your custom images. Every instruction adds a new layer to the image — understanding each one lets you write lean, fast, and reproducible builds.



FROM

Sets the base image for all subsequent instructions. E.g., `python:3.9-alpine` gives you a minimal Python runtime. Always pin a specific tag — never use `:latest` in production.



WORKDIR

Sets the working directory inside the container for all following `RUN`, `COPY`, and `CMD` instructions. Creates the directory if it doesn't exist. Keeps your image structure clean and predictable.



RUN

Executes a shell command *during the image build*. Common uses: installing packages (`apt-get install nginx`), compiling assets, or running database migrations at build time.



EXPOSE

Documents which ports the container application listens on. **Important:** EXPOSE is metadata only — it does not actually publish the port. You still need `-p` in `docker run` to bind it to the host.

CMD vs. ENTRYPOINT

This is one of the most commonly confused distinctions in Docker. Getting it right is the difference between a flexible, production-grade image and a fragile one.

CMD — Overridable Defaults

CMD provides the default command or arguments that run when the container starts. It is **easily overridden** at runtime:

```
docker run myimage echo "hello"
```

The `echo "hello"` completely replaces the CMD. Use CMD for sensible defaults that operators might reasonably change.

ENTRYPOINT — Fixed Executable

ENTRYPOINT configures the container to run as an executable. It is **not overridden** by runtime arguments — those arguments are passed *to* the entrypoint instead.

- ✓ Best Practice: Use ENTRYPOINT for the main application binary, and CMD for default flags to that binary. Together they give you a fixed executable with configurable defaults.

COPY vs. ADD

Both instructions move files from your host into the image — but they are not interchangeable. Choosing wrong introduces unnecessary complexity and security risk.

COPY — The Preferred Choice

Straightforward and transparent: copies files or directories from the build context into the container image. No hidden behavior, no side effects. Use this 95% of the time.

```
COPY ./src /app/src
```

ADD — The Power Tool

Does everything COPY does, plus two extra superpowers:

- Automatically extracts local `.tar` archives into the destination directory.
- Can fetch files from remote URLs directly into the image.

```
ADD archive.tar.gz /app/
```

 **Best Practice:** Always default to COPY. Only reach for ADD when you specifically need tar extraction. Remote URL fetching in ADD is generally discouraged — use curl/wget in a RUN command for better layer caching control.



SECTION 5

Docker Compose

The Multi-Container Problem

Why Managing Containers by Hand Breaks Down

Real-world applications are never a single container. A typical stack includes a frontend, a backend API, a database, and a caching layer like Redis. Coordinating these manually becomes a maintenance nightmare fast.

→ The Manual Approach

Four separate `docker run` commands, each with its own network flags, volume mounts, environment variables, and port mappings. Miss one flag and the whole stack breaks.

→ Ordering Problems

Your API can't start before the database is ready. Manually sequencing startup order is tedious and fragile — especially in CI environments.

→ The Solution: Docker Compose

Declare your entire multi-container application — services, networks, volumes, environment variables — in a single `docker-compose.yml` file. One command launches the whole stack: `docker-compose up -d`.

How Docker Compose Works

Docker Compose reads your YAML file and translates it into all the `docker run` commands, network configurations, and volume provisioning you'd otherwise write by hand.



Read compose file

Compose provisions

All containers running

Every service defined in Compose is automatically placed on a shared custom network, enabling DNS-based discovery between containers — no IP hardcoding required.

Anatomy of a `docker-compose.yml`

Here's a minimal but production-representative Compose file running an Nginx web server backed by a PostgreSQL database, with a named volume for persistence.

```
version: '3.8'

services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - database

  database:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

❗ The **depends_on** key ensures the `database` service starts before `web`. The named volume `db_data` persists your PostgreSQL data across container restarts and teardowns.

Key Docker Compose Commands

`docker-compose up -d`

Builds images (if needed), creates the network, provisions volumes, and starts all services in the background.

`docker-compose down`

Stops and removes all containers and the shared network. Add `-v` to also destroy named volumes.

`docker-compose logs -f`

Streams combined log output from all services. Prefix with a service name to filter: `docker-compose logs -f web`.

`docker-compose ps`

Shows the current state of all services defined in the Compose file — running, stopped, or exited.

SECTION 6

The Case for Kubernetes

Why Kubernetes?



When a Single Machine Isn't Enough

Docker Compose is great for a single laptop or server, but it stops short when applications need to run reliably at scale.



Single machine only

Compose manages containers on one host, not across a cluster.



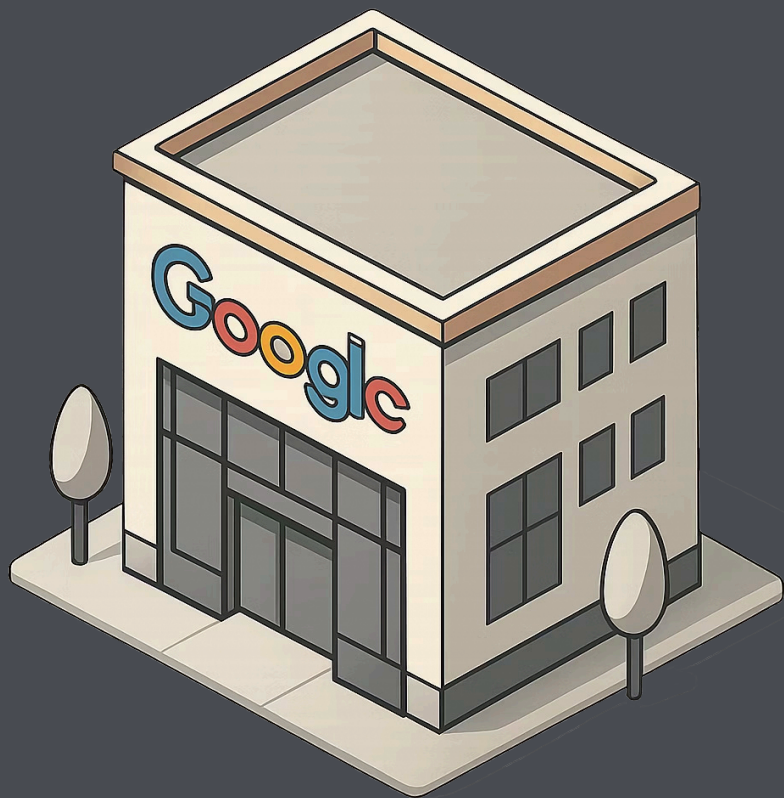
No self-healing

If a machine fails, Compose does not automatically recover those containers.



No auto-scaling

Traffic spikes still require manual intervention to add or remove capacity.



Enter Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform created by Google. It helps teams deploy, manage, and scale containerized applications across clusters of machines. Today, it is the industry standard for running containers at scale.

What Kubernetes Can Do For You



Self-Healing

Automatically recovers failed containers so your apps stay available.



Load Balancing

Spreads traffic across healthy containers to keep performance steady.



Automated Rollouts

Updates applications safely and lets you roll back if needed.



Auto-Scaling

Adjusts capacity as demand changes so your app can handle more users.

SECTION 7

How Kubernetes Works

The Big Picture



Control Plane vs. Worker Nodes

A Kubernetes cluster has two main parts that work together to keep everything running smoothly.

Control Plane (Master Node)

The Control Plane is the brain of the cluster. It makes the important decisions about what should run and where.

It keeps the whole system organized and helps the cluster stay on track.

Worker Nodes

Worker Nodes are the machines that actually run your applications.

They follow the Control Plane's decisions and do the day-to-day work.

Day 3 Summary

Docker Networking

Containers on custom networks resolve each other by container name via automatic DNS — no IP addresses needed.

Docker Volumes

Named volumes persist data beyond container lifetimes. Bind mounts accelerate local development with live code reloading.

Docker Compose

A single `docker-compose.yml` replaces complex multi-command workflows, launching an entire connected stack with one command.

Kubernetes Architecture

The Control Plane (brain) manages desired state; Worker Nodes (muscle) run your application containers. The Kubelet, API Server, Scheduler, and etcd each own a distinct responsibility.

Pods & Deployments

Pods are ephemeral compute units. Deployments manage them declaratively — maintaining replica counts, enabling rolling updates, and automatically healing failures.

Key Concepts at a Glance



Docker Compose

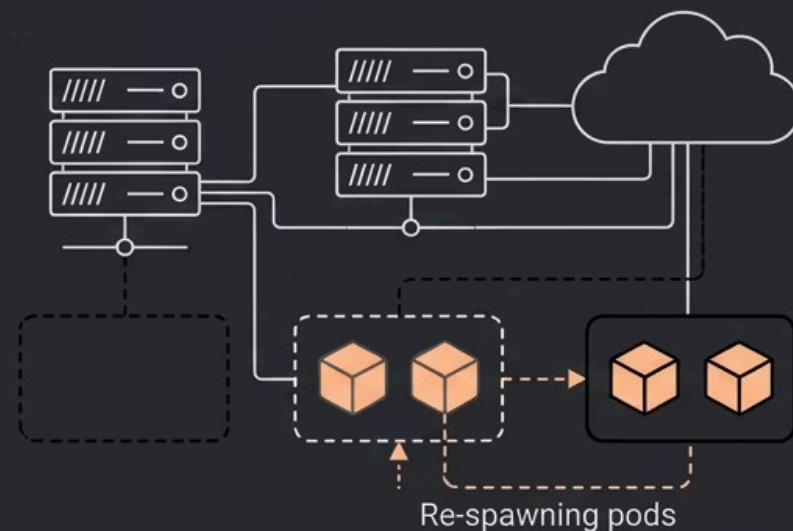
Scope: Single Machine

Self-healing: No

Scaling: Manual

Networking: Custom Bridge DNS

Use Case: Local Dev



Kubernetes

Scope: Multi-server Cluster

Self-healing: Yes, Auto

Scaling: Automated

Networking: Services w/ IPs

Use Case: Production Scale

Next Steps — Your Day 3 Action Items

1

Multi-Container Lab

Complete the Docker Compose lab — wire up a frontend, backend API, and PostgreSQL database using a single `docker-compose.yml`. Verify service-to-service DNS resolution is working.

2

Volume Practice

Protect a PostgreSQL database with a named volume. Destroy and recreate the container, then verify your data survived. Try a bind mount for a Node.js app and confirm live-reload is working.

3

Advanced Dockerfile

Write a Dockerfile that uses both `ENTRYPOINT` and `CMD` correctly. Experiment with overriding `CMD` at runtime. Refactor any existing Dockerfiles that use `ADD` unnecessarily.

4

Install Minikube

Get your local Kubernetes environment ready for tomorrow's intensive K8s labs. Run `minikube start` and confirm `kubectl get nodes` returns a healthy node. Don't skip this step — labs depend on it.

✔ Tomorrow we go deep on K8s Services, ConfigMaps, Secrets, Persistent Volumes, and deploying a full application to your local cluster. Come prepared!

Thank You!

We sincerely appreciate your time and engagement during Day 3, where we covered essential concepts of Docker Compose and an introduction to Kubernetes.

See you on Day 4!

